Statistical Machine Learning Coursework

Calum Regan 11013166

March 2026

1 Task 1: K-Nearest Neighbours on Blood Cell Anomaly Data

K-Nearest-Neighbours is a supervised learning technique that uses the distance between points to make a prediction given their K nearest neighbours. For a given point $\mathbf{x}$, the set of indices of K observations is the indices of the K closest data points. The most common class or label in the K nearest observations is what is given to $\mathbf{x}$. This is the KNN rule $f_{KNN}(\mathbf{x})$. Any distance metric can be used but the most common is the Euclidean metric.

If we split out data into two subsets, training $\mathcal{T}$ and testing $\mathcal{S}$, the accuracy of the model can be calculated by

$$Accuracy(f_{KNN}(\mathbf{x})) = \frac{1}{|S|}\sum_{i\in\mathcal{S}} 1_{\{f_{KNN}(\mathbf{x};K,\mathcal{T})=y_i\}}$$

where $1_{\{\}}$ is the indicator function. In words, the accuracy is is calculated as the number of times the KNN rule correctly predicts the label of $\mathbf{x}$ in the testing dataset out of the total number of points in the data set. K is a hyperparamter of the model that can be tuned to fit the data better. In this task, the test accuracy will calculated for a range of K and the model with the highest accuracy will be selected.

The data for this method is a CSV file from Kaggle at www.kaggle.com/datasets/alitaqishah/blood-cell-anomaly-detection-2025. It is about blood cell anomalies and contains 5,880 blood cell records over 19 cell types with a variety of different biomarkers such as cell diameter or cytoplasm ratio. The aim in this task is to predict what type of cell a sample is.

The first step of preprocessing was to drop columns that would not affect the outcome such as dataset source, microscope model or cell id. Secondly, the data has a variety of scales and since KNN works with distances between points, we need to scale the data to prevent the different scales affecting the results. `StandardScaler()` from `Scikit-Learn` is the type of scalar used to which standardises the data to mean 0 and variance 1. Finally, the data was split into training and testing sets with a 80/20 split.

The code for this method is given below.

```
#define features by dropping the label and other irrelevnt features
X = data.drop(['cell_id', 'cell_type', 'dataset_source', 'disease_category',
                'staining_protocol', 'microscope_model', 'patient_age_group',
               'patient_sex', 'labeller_confidence_score'], axis=1)
y = data["cell_type"] #define label which is cell type
scalar = StandardScaler() #different scales ruins accuracy
X_scaled = scalar.fit_transform(X) #standardise features to have mean 0, variance 1

train_acc = [] #train accuracy storage
test_acc = [] #test accuracy storage
for k in range(1, 25): #run kNN with different values of k
    knn = KNeighborsClassifier(n_neighbors=k) #classifier with k nearest neighbours
    #split data into training and testing subsets
    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=730
    knn.fit(X_train, y_train) #train model on the training data
    #add the training and testing accuracies to the storage
    train_acc.append(round(knn.score(X_train, y_train), 4))
    test_acc.append(round(knn.score(X_test, y_test), 4))
#plot training, test accuracy, select the best k
knn_best = KNeighborsClassifier(n_neighbors=15) #classifier with best k
knn_best.fit(X_train, y_train) #fit this best classifier
y_pred = knn_best.predict(X_test) #predict y labels
cm = confusion_matrix(y_test, y_pred) #form confusion matrix
#plot confusion matrix
```

Figure 1 shows the accuracy of the training and testing for different values of K and Figure 2 shows the confusion matrix of the most accurate value of K in testing.

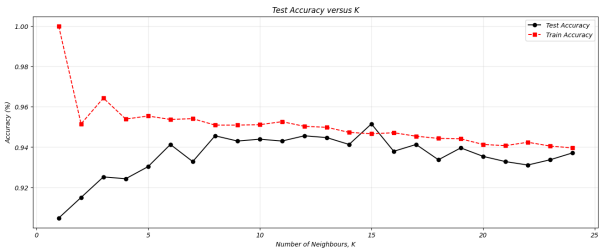


Figure 1: Training and Test Accuracy of Different Values of K

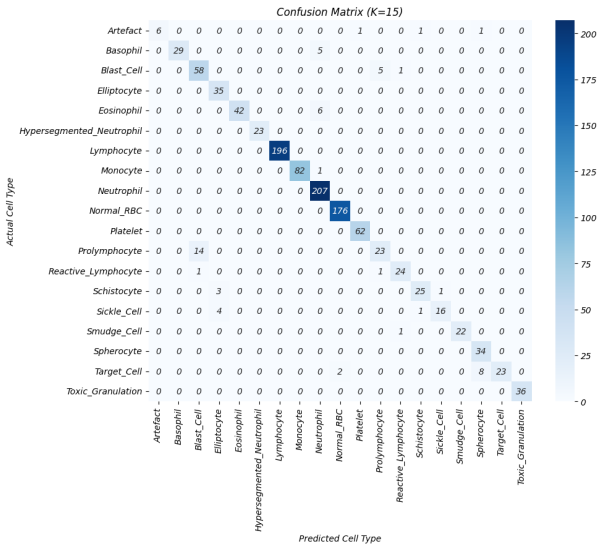


Figure 2: Confusion Matrix for 9-Nearest-Neighbours

From Figure 1, the model with the highest test accuracy at 95.15% is when $K = 15$. $K = 15$ also has a high training accuracy at 94.66%. There is only a small difference between these accuracies which suggest there is not over fitting because the model is performing just as well on test data as it did on training data. This is not the case for $K = 8$ and below where test accuracy is much lower but training accuracy is significantly higher. This means the model for these values of K suffers from over-fitting, essential memorising the training data and then failing to accurately predict the test data. There is also a weak downwards trend in both test and training accuracy past $K = 15$. This is because the larger K is, the more neighbours a prediction is based on, some of which could be quite far away and not similar to $\mathbf{x}$, increasing the prediction error and lowering accuracy.

The confusion matrix in Figure 2 visually shows the predictions the model gets correct or incorrect for the most accurate model $K = 15$. The majority of the matrix is 0 and the high counts are concentrated on the diagonal which confirms the fact that we have a high accuracy. The matrix does offer more insight into where the classifier is failing. Taking the entry with the most incorrect predictions, we can see Pro-lymphocytes are misclassified as Blast cells 14 times. This is because both cells are types of lymphoid cells with similar but different functions meaning the model sometimes struggles to distinguish the small differences between them. However, this problem is only really a small issue for these types of cells and in general, a 15-NN model is an effective model for predicting cell type in this dataset.

2 Task 2: Random Forest on Loan Approval Data

Random forest is a collection of decision trees that uses averaging over bootstrap samples to improve predictions and reduce over-fitting. In a regular decision tree, a yes/no question or criterion is asked at the root and non-terminal nodes which splits the data into 2 parts. Questions keep being asked until the tree reaches a terminal node. The goal of the decision tree is to create a node with a low Gini coefficient which is given by

$$Gini(r) = \sum_{y=1}^{m} \hat{p}_{r,y}(1 - \hat{p}_{r,y})$$

where $\hat{p}_{r,y}$ is the proportion of observations with label y in the r-th node. A low Gini coefficient indicates that a node is made up of most of the same class. The depth of the tree is a hyper-parameter which can be tuned to best fit the data.

In a random forest, the data is sampled and replaced to form bootstrap samples. Then, a number of different decision trees are fit on the bootstrap samples, with a random selection of features used at each split, and the modal class is taken as the prediction in the classification setting.

The data is a CSV from www.kaggle.com/datasets/algozee/credit-risk-and-loan-default-analysis-dataset. It is contains features such as age, income and credit score for individuals applying for loans. There is also a column for whether the loan is approved or not. The aim of this task is to predict if an individual was approved for a loan or not.

Very little preprocessing was required. The LoanApproved column was dropped to define the feature matrix and then the categorical variables like gender, employment and education were encoded to integers. Finally, the data was split into training and testing sets with a 80/20 split.

The code for this method is given below.

```
#encode categorical variables to integerers
data_loan_encoded = pd.get_dummies(data_loan, drop_first=True)
#define features and labels
X_loan = data_loan_encoded.drop('LoanApproved', axis=1)
y_loan = data_loan_encoded['LoanApproved']
#split into training and test sets
X_train_loan, X_test_loan, y_train_loan, y_test_loan = train_test_split(
    X_loan, y_loan, test_size=0.2, random_state=472)
#define and fit random forest classifier
rf = RandomForestClassifier(n_estimators=200,
                            max_depth=8,
                            oob_score=True,
                            random_state=472)
rf.fit(X_train_loan, y_train_loan)
#accuracies
print('Train Accuracy:', round(rf.score(X_train_loan, y_train_loan), 4))
print('Test Accuracy:', round(rf.score(X_test_loan, y_test_loan), 4))
#get names of features and how important they are
feature_names = X_train_loan.columns
importances = rf.feature_importances_
```

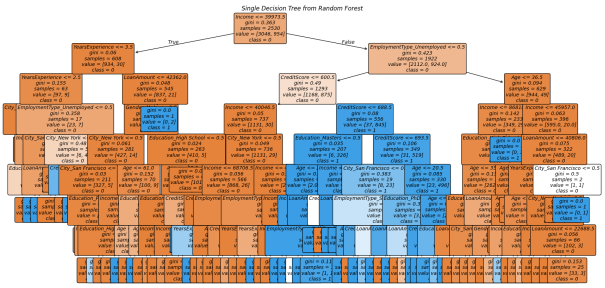


Figure 3: A Single Tree from Random Forest Model

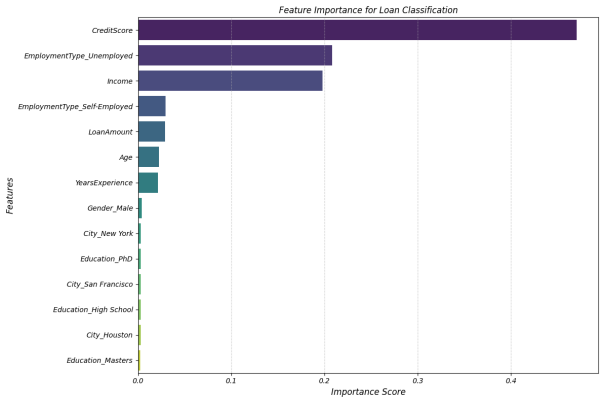


Figure 4: Feature Importance for Random Forest

The training accuracy for this model is 97.22% and the testing accuracy is 96.80%. Both are very high and close in value which suggests the model is not over-fitting on the training data and can accurately predict if a loan is approved or not on the testing dataset.

In the code, maximum tree depth is chosen at 8. Any less will fail to properly classify points into each category but a higher max depth would result in over-fitting and the model performing poorly on testing data. 200 estimators means 200 individual trees were fitted. For more than 200 estimators, there was no improvement in accuracy but a longer computational time and fewer than 100 resulted in a lower accuracy score because the model had less predictions to average over.

Figure 3 shows what 1 tree in the random forest model looks like. Each node on the tree shows the criteria on which the split in the node occurs. This is done by the Gini coefficient and the model class at that node.

Figure 4 shows how important each feature is in the model. Clearly, credit score is the most significant feature in determining if a loan is approved or not. Other important features are if an individual is unemployed and their income. This follows the logic in real life when applying for a loan as your income, and employment status are big indicators on if an individual can make repayments and credit score shows your past credit history. The insignificant features are city and education level which also makes sense since loan approval is a financial decision, not a location-based one.

3 Task 3: HDBSCAN on Gaia Stars Dataset

UMAP stands for Uniform Manifold Approximation and Projection and is a dimension reduction technique. It assumes the data lies on a low dimensional manifold in high dimensional space. Firstly, for each data point we find the distance to the nearest neighbour, denoted ρ_i. We also have weights ω_{ij} for each observation with an associated σ_i which controls how large the neighbourhood around the point is and is governed by the number of nearest neighbours. This forms an undirected graph. Next, the distances between the points are calculated in the lower dimension, these distances also have weights and a similar undirected graph is constructed in the lower dimension. Finally, a cost function based on cross-entropy is minimised by adjusting the points in low dimensional space.

HDBSCAN stands for Hierarchical Density-Based Spatial Clustering of Applications with Noise and is a type of unsupervised learning. The goal is to find high density clusters separated by low density regions. Firstly, we find the distance between a data point and its k-th nearest neighbours. Distances between data points are redefined so that points close together are kept close while points that are far away and given large distances making them easier to separate and are then dropped as noise. This means we can have clusters of different densities. Minimum cluster size is a hyper-parameter that defines the minimum amount of data points for a group of points to be considered a cluster. Minimum sample size is another hyper-parameter that defines the minimum amount of nearest neighbours a point must have to not be considered noise.

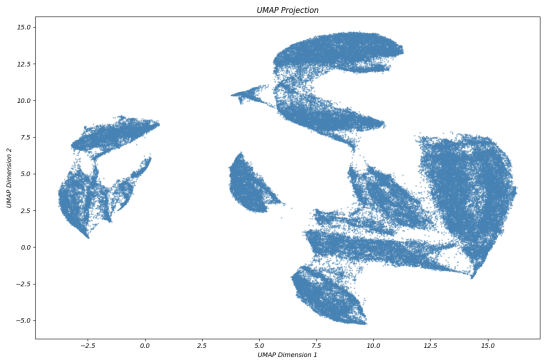
This data is a CSV file from www.kaggle.com/datasets/solorzano/rave-dr5-gaia-dr2-consolidated. It contains data collected from the European Space Agency (ESA) on 250,000+ stars. The features include information on the speed, direction, location and mass of individual stars. There 36 columns in this dataset. The aim of this clustering task is to identify clusters of stars with similar properties.

The data must be processed before the HDBSCAN can be performed. The data had over 250,000 rows with 36 columns so to help with the runtime of the code, 100,000 samples were taken from the original dataset. Next, the samples were scaled using `StandardScaler()` so the values have mean 0 and variance 1 because there are a wide range of scales between the columns. The data still has a high dimension so we will use UMAP for dimension reduction to 2 dimensions. Then, HDBSCAN will be applied to identify clusters in the data.

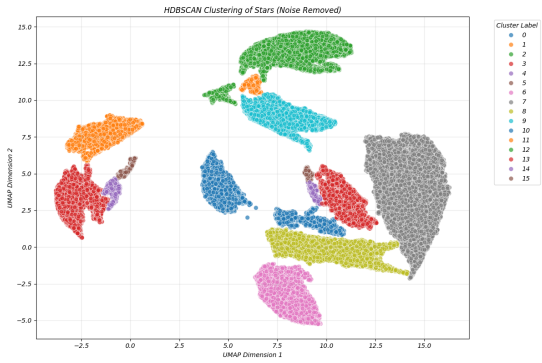
The code for this method is shown below.

```
#data is 250,000+ rows so sample 100000
stars_sample = stars.sample(100000, random_state=730).reset_index(drop=True)
#different features have different scales
#standardise them to have mean 0 and variance 1
stars_scaled = scalar.fit_transform(stars_sample)
#define umap dimension reduction to reduce the data to 2d for visualisation
reducer = umap.UMAP(n_components=2, n_jobs=1,random_state=730)
#perform umap dimension reduction on the scaled data
stars_umap = reducer.fit_transform(stars_scaled)
#define and fit the HDBSCAN clustering algorithm on the UMAP reduced data
hdb = HDBSCAN(min_cluster_size=300,
              min_samples=10,
              cluster_selection_method='eom'
              )
hdb.fit(stars_umap)
#get the cluster labels assigned by HDBSCAN
cluster_labels = hdb.labels_
```

Figure 5b shows the clusters that the HDBSCAN identifies in the sample of the dataset.



(a) UMAP Dimension Reduction for Gaia Stars



(b) HDBSCAN for Gaia Stars

Figure 5: UMAP to HDBSCAN

From the code, when we have a larger minimum cluster size, clusters tend to get merged together and we lose some of the underlying structure in the data. Similarly, a minimum cluster size that is too small results in very small clusters that breaks up the data too much so the clustering becomes useless and unreadable in a graph.

From Figure 5a, the UMAP dimension reduction to 2 dimensions allows us to plot the data set to see it's structure. The data has been grouped as it has been transformed from high to low dimensional space. There is a small collection of fragmented clusters on the left which is distinctly separated from a larger collection of interconnected clusters on the right . Applying HDBSCAN to this lower dimensional data will reveal if there are any other clusters within a larger group that have not been identified.

Figure 5b shows what UMAP could not. HDBSCAN identified 16 clusters and some noise labelled -1. The model has largely followed the structure from the UMAP process but uncovered some additional structures. The distinct group on the right in Figure 5a has been divided into multiple clusters. Even the smaller group on the left in Figure 5a has been split up into distinct clusters. The noise has been removed from Figure 5b to show the clear boundaries between groups. The noise points occur when a data point has less than the minimum number of neighbouring samples, so cannot be assigned to a cluster.

These clusters could represent a number of different things. They could represent star types, stars that share similar masses and sizes or stars that move with similar directions and speeds.

Since only 100,000 stars were sampled from the whole 250,000+ and the data was reduced from 36 to 2 dimensions, this combination means some clusters may have been or not properly identified.